Symulator programowy mikroprocesora

Projekt z przedmiotu Organizacja Systemów Komputerowych - laboratorium

Michał Kossakowski 197589 Hubert Kowalski 197811

13 marca 2026

Streszczenie

Niniejsze sprawozdanie opisuje projekt symulatora prostego mikroprocesora z czterema 16-bitowymi rejestrami ogólnego przeznaczenia, zrealizowany w języku Python z graficznym interfejsem użytkownika. Program umożliwia pisanie, edycję, zapis i odczyt programów w asemblerze, a także ich wykonywanie w trybie ciągłym lub krokowym z wizualizacją stanu rejestrów. W dokumencie przedstawiono przyjęte założenia, kluczowe rozwiązania programistyczne oraz dyskusję nad osiągniętymi wynikami.

1 Sformułowanie zadania i przyjęte założenia

1.1 Cel projektu

Celem projektu było stworzenie programowego symulatora mikroprocesora, który umożliwia:

- Symulację procesora wyposażonego w cztery 16-bitowe rejestry ogólnego przeznaczenia: AX, BX, CX, DX.
- Traktowanie każdego rejestru jako pary 8-bitowych rejestrów: NH (starszy bajt) i NL (młodszy bajt), gdzie N to A, B, C lub D.
- Obsługę trzech rozkazów: MOV (przesłanie), ADD (dodawanie), SUB (odejmowanie).
- Realizację dwóch trybów adresowania: rejestrowego i natychmiastowego.
- Możliwość pisania krótkich programów z użyciem dostępnych rozkazów.
- Wykonanie programów w trybie całościowym oraz krokowym z podświetleniem aktualnie wykonywanej instrukcji.
- Zapis i odczyt programów do pliku tekstowego.

1.2 Przyjęte założenia szczegółowe

W celu jednoznacznego określenia działania symulatora przyjęto następujące założenia:

- **Składnia instrukcji:** Każda instrukcja zapisywana jest w osobnej linii. Dopuszczalny jest przecinek jako separator argumentów (np. `MOV AX, BX`) lub jego brak (np. `ADD AX 5`). Wielkość liter nie ma znaczenia.
- **Komentarze:** Linie rozpoczynające się od średnika (;) są ignorowane podczas wykonywania.
- **Rejestry:** Dozwolone nazwy rejestrów to: AX, AH, AL, BX, BH, BL, CX, CH, CL, DX, DH, DL. Odwołanie do rejestru 8-bitowego wpływa tylko na odpowiednią część rejestru 16-bitowego.
- **Liczby:** Argumenty natychmiastowe podawane są w systemie dziesiętnym (np. `SUB CX, 10`). W przypadku nadmiaru (wynik poza zakresem) wartość jest zawijana modulo odpowiedni rozmiar (16 bitów – 0–65535, 8 bitów – 0–255).
- **Pusta linia:** Puste linie są pomijane przy wykonywaniu.
- **Tryb krokowy:** Po każdym kroku program zatrzymuje się, a użytkownik musi potwierdzić chęć wykonania kolejnej instrukcji. Bieżąca linia jest podświetlona na żółto.

2 Opis przyjętych rozwiązań programowych

Symulator został napisany w języku Python z wykorzystaniem biblioteki Tkinter do budowy graficznego interfejsu użytkownika. Program składa się z jednej klasy `SimulatorGUI`, która zawiera całą logikę symulatora oraz elementy interfejsu. Poniżej przedstawiono kluczowe fragmenty kodu ilustrujące najważniejsze mechanizmy.

2.1 Mapowanie rejestrów 8-bitowych na 16-bitowe

Podstawowym problemem było zapewnienie spójnego dostępu do rejestrów 16-bitowych i ich 8-bitowych składowych. Zastosowano słownik mapujący każdą dozwoloną nazwę na krotkę (baza, część), gdzie `baza` to jeden z rejestrów 16-bitowych (AX, BX, CX, DX), a `część` przyjmuje wartość `None` (dla 16-bitowego), `'high'` (dla starszego bajtu) lub `'low'` (dla młodszego bajtu).

```
REG_MAP = {
    'AX': ('AX', None), 'AH': ('AX', 'high'), 'AL': ('AX', 'low'),
    'BX': ('BX', None), 'BH': ('BX', 'high'), 'BL': ('BX', 'low'),
    'CX': ('CX', None), 'CH': ('CX', 'high'), 'CL': ('CX', 'low'),
    'DX': ('DX', None), 'DH': ('DX', 'high'), 'DL': ('DX', 'low'),
}
```

Listing 1: Mapowanie nazw rejestrów

Funkcje `get_reg` i `set_reg` wykorzystują to mapowanie do odczytu i zapisu wartości z uwzględnieniem odpowiedniego bajtu.

```python
def get_reg(self, name):
    base, part = self.REG_MAP[name.upper()]
    val = self.regs[base]
    if part is None:
        return val & 0xFFFF
    elif part == 'high':
        return (val >> 8) & 0xFF
    else:
        return val & 0xFF

def set_reg(self, name, value):
    base, part = self.REG_MAP[name.upper()]
    if part is None:
        self.regs[base] = value & 0xFFFF
    else:
        current = self.regs[base]
        if part == 'high':
            new = (current & 0x00FF) | ((value & 0xFF) << 8)
        else:
            new = (current & 0xFF00) | (value & 0xFF)
        self.regs[base] = new & 0xFFFF
```

Listing 2: Odczyt i zapis rejestrów

2.2 Parsing linii programu

Funkcja `disassemble` odpowiada za analizę pojedynczej linii kodu. Usuwa przecinki, dzieli na tokeny i sprawdza poprawność rozkazu oraz rejestru docelowego. Źródło jest walidowane później, w funkcji `parse_operand`.

```python
def disassemble(self, line):
    line = line.replace(',', ' ').strip()
    if not line or line.startswith(';'):
        return None
    parts = line.split()
    if len(parts) < 3:
        raise ValueError("Za ma o argument w")
    op = parts[0].upper()
    if op not in ('MOV', 'ADD', 'SUB'):
        raise ValueError(f"Nieznana instrukcja: {op}")
    dest = parts[1].upper()
    src = parts[2].upper()
    if dest not in self.VALID_REGS:
        raise ValueError(f"Nieprawid owy rejestr docelowy: {dest}"
            )
    return (op, dest, src)
```

Listing 3: Parsowanie instrukcji

2.3 Wykonanie instrukcji

Metoda `execute` realizuje właściwą operację na rejestrach. Najpierw pobierana jest wartość źródła (z rejestru lub bezpośrednia), następnie wykonywane jest działanie, a na końcu wynik jest zapisywany z uwzględnieniem rozmiaru rejestru docelowego.

```
def execute(self, op, dest, src):
    src_type, src_val = self.parse_operand(src)
    if src_type == 'reg':
        src_val = self.get_reg(src_val)
    dest_val = self.get_reg(dest)
    if op == 'MOV':
        result = src_val
    elif op == 'ADD':
        result = dest_val + src_val
    elif op == 'SUB':
        result = dest_val - src_val
    else:
        raise ValueError(f"Nieznana operacja: {op}")
    base, part = self.REG_MAP[dest]
    if part is None:
        result &= 0xFFFF
    else:
        result &= 0xFF
    self.set_reg(dest, result)
```

Listing 4: Wykonanie rozkazu

2.4 Tryb krokowy

Realizacja trybu krokowego opiera się na metodzie `step`, która wykonuje pojedynczą instrukcję (pomijając puste linie i komentarze) i aktualizuje podświetlenie w edytorze. Pętla w interfejsie oczekuje na naciśnięcie przycisku „Krok” i wywołuje tę metodę.

```
def step(self):
    self.clear_error()
    if self.pc < 0 or self.pc >= len(self.program):
        self.log_error("Brak programu lub koniec programu.")
        self.running = False
        return False

    # Pomi  puste linie i komentarze
    while self.pc < len(self.program):
        line = self.program[self.pc].strip()
        if line and not line.startswith(';'):
            break
        self.pc += 1

    if self.pc >= len(self.program):
        self.log_error("Koniec programu.")
        self.running = False
        return False
```

```

    line = self.program[self.pc].strip()
    try:
        op, dest, src = self.disassemble(line)
        self.execute(op, dest, src)
        self.pc += 1
        self.update_program_display()
        return True
    except Exception as e:
        self.log_error(f"B   d  w linii {self.pc+1}: {e}")
        self.running = False
        self.update_program_display()
        return False
```

Listing 5: Wykonanie jednego kroku

2.5 Wizualizacja stanu

Stan rejestrów jest na bieżąco aktualizowany w interfejsie po każdej zmianie. Funkcja `update_register_display` odświeża etykiety z wartościami dziesiętnymi i szesnastkowymi. Bieżąca linia programu jest podświetlana przez dodanie tagu `current` w polu tekstowym.

```
def update_program_display(self):
    self.program_text.tag_remove(self.current_line_tag, '1.0', tk.END)
    if self.pc >= 0 and self.pc < len(self.program):
        line_start = f"{self.pc+1}.0"
        line_end = f"{self.pc+1}.end"
        self.program_text.tag_add(self.current_line_tag, line_start, line_end)
        self.program_text.tag_config(self.current_line_tag, background='yellow', foreground='black')
```

Listing 6: Aktualizacja podświetlenia

2.6 Zapis i odczyt z pliku

Program korzysta z natywnych okien dialogowych Tkinter do wyboru pliku. Zawartość pola edycyjnego jest zapisywana w czystej postaci tekstowej, co zapewnia czytelność i możliwość edycji w zewnętrznych edytorach.

```
def save_file(self):
    fname = filedialog.asksaveasfilename(defaultextension=".txt")
    if not fname:
        return
    try:
        content = self.program_text.get('1.0', tk.END)
        with open(fname, 'w', encoding='utf-8') as f:
            f.write(content)
        messagebox.showinfo("Info", "Program zapisany.")
```

```
    except Exception as e:
        messagebox.showerror("B   d", f"Nie mo na zapisa  pliku
           :\n{e}")
```

Listing 7: Zapis do pliku

3 Dyskusja osiągniętych wyników

3.1 Zalety aplikacji

- **Przyjazny interfejs graficzny** – aplikacja jest intuicyjna, nawet dla osób niemających doświadczenia z symulatorami procesorów. Wszystkie elementy są czytelnie oznaczone, a stan rejestrów wyświetlany w przejrzysty sposób.
- **Pełna funkcjonalność** – program realizuje wszystkie wymagania postawione w zadaniu: obsługę rejestrów 16- i 8-bitowych, trzy rozkazy, dwa tryby adresowania, pracę ciągłą i krokową, zapis/odczyt z pliku.
- **Wizualizacja** – podświetlanie bieżącej linii w trybie krokowym oraz dynamiczna aktualizacja rejestrów ułatwiają śledzenie przebiegu programu i zrozumienie działania procesora.
- **Obsługa błędów** – komunikaty o błędach są wyświetlane w wydzielonym polu, a nie w oknach dialogowych, co nie przerywa pracy i pozwala na szybką korektę.
- **Przenośność** – Python i Tkinter działają na wszystkich głównych systemach operacyjnych (Windows, Linux, macOS).

3.2 Wady i ograniczenia

- **Ograniczona lista rozkazów** – symulator wspiera tylko trzy instrukcje. Brakuje podstawowych operacji jak skoki, porównania czy operacje logiczne, co uniemożliwia pisanie bardziej złożonych programów.
- **Brak wsparcia dla etykiet i skoków** – program jest wykonywany liniowo, nie ma możliwości implementacji pętli ani warunków.
- **Tylko liczby dziesiętne** – brak możliwości podawania argumentów w systemie szesnastkowym (np. 0x10) może być uciążliwy dla osób przyzwyczajonych do asemblera.
- **Prosty edytor** – edytor wbudowany w aplikację oferuje podstawowe funkcje; brak zaawansowanych opcji jak automatyczne kolorowanie składni, podpowiedzi itp.
- **Wydajność** – przy bardzo długich programach (tysiące linii) interfejs mógłby działać nieco wolniej, choć w typowych zastosowaniach dydaktycznych nie stanowi to problemu.

3.3 Możliwości rozwoju

Aplikację można rozbudować o:

- Dodanie kolejnych rozkazów (np. MUL, DIV, AND, OR, JMP, CMP).
- Implementację etykiet i skoków warunkowych.
- Obsługę liczb szesnastkowych (prefiks 0x lub h).
- Zapis i odczyt stanu symulatora (rejestry, licznik rozkazów).
- Kolorowanie składni w edytorze.
- Możliwość ustawiania breakpointów.

4 Podsumowanie

Stworzony symulator mikroprocesora spełnia wszystkie założenia projektowe i stanowi użyteczne narzędzie dydaktyczne do nauki podstaw programowania w asemblerze oraz zrozumienia działania rejestrów i trybów adresowania. Dzięki graficznemu interfejsowi jest łatwy w obsłudze, a tryb krokowy z wizualizacją pozwala na dogłębną analizę wykonywanego kodu. Mimo pewnych ograniczeń, aplikacja z powodzeniem może być wykorzystywana podczas zajęć laboratoryjnych z organizacji systemów komputerowych.

Literatura

[1] Python Software Foundation. *Python 3 Documentation*. `https://docs.python.org/3/`

[2] Tkinter — Python interface to Tcl/Tk. `https://docs.python.org/3/library/tkinter.html`

[3] Intel 64 and IA-32 Architectures Software Developer Manuals.